

Approach — Conversational SHL Assessment Recommender

1. Problem framing

The task is not “search a catalog”; it is running a *coherent multi-turn dialogue* that decides, each turn, whether to **clarify, recommend, refine, compare, or refuse** — and stays grounded in the SHL Individual Test Solutions catalog. I treated the 10 provided traces as the behavioral spec and reverse-engineered the contract from them before writing the agent:

- Clarify vs. recommend is a *decision*, not a fixed step: C1 clarifies twice on a vague “senior leadership” opener; C4/C8/C10 recommend on turn 1 because the role is already concrete.
- Honesty about catalog gaps: C2 (no Rust test) and C7 (English-only knowledge tests) must surface the constraint and offer the closest real items — never fabricate.
- Grounded compare (C3/C5/C6), refine-in-place (C9/C8/C4), scope refusal for legal questions mid-conversation while keeping the shortlist (C7), and principled push-back that still honors the user’s final call (C10).
- OPQ32r recurs as a near-universal personality default.

2. System design

POST /chat runs one orchestrator pass: **guard** → **retrieve** → **one LLM call** → **ground** → **enforce**. The API is stateless; every call rebuilds context from the full history, which is precisely what makes refinement work — “drop REST, add Docker” re-derives the candidate pool from all accumulated constraints, so the shortlist updates instead of resetting.

Grounding boundary (anti-hallucination). The LLM never emits URLs. It receives a candidate pool (retrieved catalog slice) and may only return `selected_ids` from it; every id is then re-resolved against the catalog and anything unresolved is dropped. If a “recommend” grounds to zero items, the agent fails *safe* to a clarify rather than presenting an empty shortlist. This makes “items from catalog only” a structural guarantee.

Turn budget. `turns_remaining` is passed into the prompt; at ≤ 2 remaining the agent is instructed to commit to a shortlist rather than keep clarifying, so it never burns the 8-turn cap asking questions.

Resilience. Catalog load degrades live → disk cache → bundled snapshot. LLM/JSON failures fall back to a deterministic decision. A top-level handler keeps every response schema-valid and HTTP 200 — a recommender that 500s on cold start scores zero.

3. Catalog & retrieval

The catalog is ingested live from the provided JSON endpoint and normalized defensively. A real detail the schema forced: the live feed ships only human-readable `keys` (e.g. “Knowledge & Skills”) and no test-type *code*, so the loader derives the `test_type` code string from those labels (GSA’s [“Competencies”, “Knowledge & Skills”] → “C,K”, matching the traces). Ingestion also handles field aliases (`link` → `url`), `remote/adaptive` yes/no flags, URL-derived stable ids, and a Job-Solution filter (Individual Test Solutions only). Load degrades live → on-disk cache → bundled snapshot, so cold start never fails.

Retrieval is **lexical-first hybrid**: BM25 over name+description+keys, plus a fuzzy name boost (`opq` → `OPQ32r`). This is deliberate — in the traces the decisive signal is named-skill overlap (“Docker” → Docker test), which lexical methods nail with zero model-download and zero per-query latency (important under the 30s cap and free cold-start hosting). A dense-embedding path (`all-MiniLM-L6-v2`) fuses in when enabled, to bridge vocabulary gaps (“plant operator” → “safety & dependability”); it is optional and the service runs identically without it. Multi-skill JDs are handled by `multi_search`: each skill fragment is queried separately and the per-item best score kept, so one test surfaces per skill instead of a blurred blend.

4. Prompt & context engineering

A single system prompt encodes the trace contract (clarify/recommend thresholds, refine-in-place, catalog-only compare, scope refusal, OPQ default). The user message is a compact JSON context: history, `turns_remaining`, and the candidate pool (id, name, type, keys, duration, languages, 240-char description). Structured JSON output (`action`, `reply`, `selected_ids`, `end_of_conversation`) is parsed tolerantly (code-fence stripping, first-object extraction) and validated against an allowed action set.

5. LLM & stack choices

FastAPI + Pydantic (strict schema at the boundary). The LLM client is OpenAI-compatible so Groq / OpenRouter / OpenAI / local are interchangeable; Groq is the default for low latency inside the 30s window with a single call per turn. A dependency-free `StubLLM` reproduces the trace behaviors heuristically so the entire service — routing, grounding, guards, schema, turn-cap — runs and is unit-tested with **no API key**. I avoided a heavy agent framework: the control flow is small and explicit, which keeps it debuggable and defensible.

6. Evaluation

A local replay harness (`app/eval/replay.py`) mirrors the grader’s shape and reports the four dimensions the brief asks for. I built 10 personas directly from the traces. Offline (retrieval-only stub, no LLM reasoning) it reports: **recommendation relevance — mean Recall@10 = 0.60**; **groundedness — 100% of emitted recommendations resolve to a real catalog item** (verifying the grounding boundary end-to-end); **response accuracy — behavior probes 4/4** (no-recommend on vague turn 1, off-topic refusal, injection refusal, grounded recommend); and **retrieval quality** — I confirmed the relevant items sit in the retrieved pool even for the low-recall personas (e.g. C6), so those misses are selection gaps the stub can’t narrow, not retrieval gaps. The 0.60 is a conservative floor. `tests/test_agent.py` covers the hard-eval surface (schema, catalog-only recs, ≤ 10 cap, refusals carrying empty recs, routing, graceful degradation) — 11/11 passing offline. With a real LLM (Groq `llama-3.3-70b-versatile`) wired in, per-turn behavior — clarify/recommend/refine/compare/refuse — is qualitatively better (natural, reasoned replies; correctly handles catalog gaps by saying so instead of padding); however a *full-batch* replay run against Groq’s free tier hit 429 rate limits partway through, silently degrading later personas to the stub fallback and dragging the measured mean recall down to 0.57 and probes to 2/4 — see §7.

7. What didn't work / trade-offs

- **Pure semantic retrieval** over-retrieved generic personality tests for specific skill queries and added model-download cost/latency; demoting it to an optional fusion layer behind BM25 fixed both precision and cold-start.
- **Letting the model return names/URLs freely** invited hallucinated URLs; the id-pool + re-resolution boundary removed that failure class entirely.
- **Padding to 10** hurt precision on the trace-style shortlists (which are 2–7 items); the prompt now prefers targeted lists.
- **Groq free-tier rate limits during batch eval.** Running the full 10-persona × 8-turn replay back-to-back against Groq's free tier triggers 429s partway through; the orchestrator's LLM-failure fallback keeps every response schema-valid (no crash), but it silently swaps in the deterministic stub for the rate-limited turns, which *understated* the real-LLM numbers in a batch run rather than overstating them. Single interactive turns (the actual product usage pattern) are unaffected — confirmed live post-deploy. For a clean full-batch measurement, the harness would need inter-call throttling or a paid tier; I did not implement this since it doesn't affect the served product's behavior, only back-to-back scripted eval runs.

8. AI-tool usage

Used an AI coding assistant to scaffold boilerplate (Pydantic models, FastAPI wiring, deploy configs) and to draft docstrings. All design decisions — the grounding boundary, stateless-refine strategy, lexical-first retrieval, turn-budget logic, and the degradation ladder — are mine and are reflected directly in the code structure.